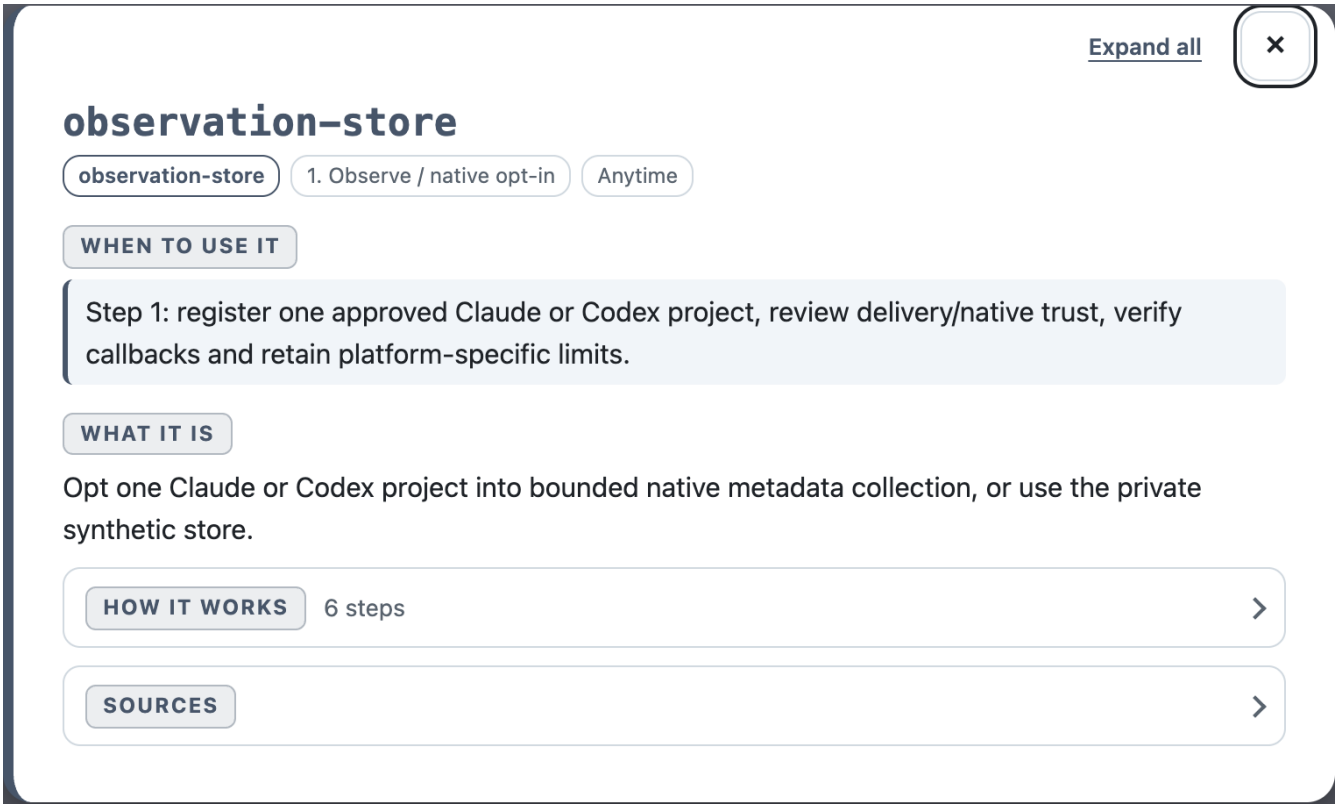
Opt in to Codex observations

Scenario: native-observation / workflow-map. 2026-10-18. A committed source, Python 3.11+, Codex and an explicitly authorized disposable project are required. Collection is off by default. Native trust and model access are separate prerequisites.

1. Choose one client and review trust

Open the map: Anytime > Self-improvement and eval > 1. Observe. Follow the native guide to create one exact-project Codex grant and preview its installer registration. Review the four observer definitions in native /hooks before activation; existing planning trust is separate.



Actual current map dialog. Follow its source guide for the authoritative commands and limits.

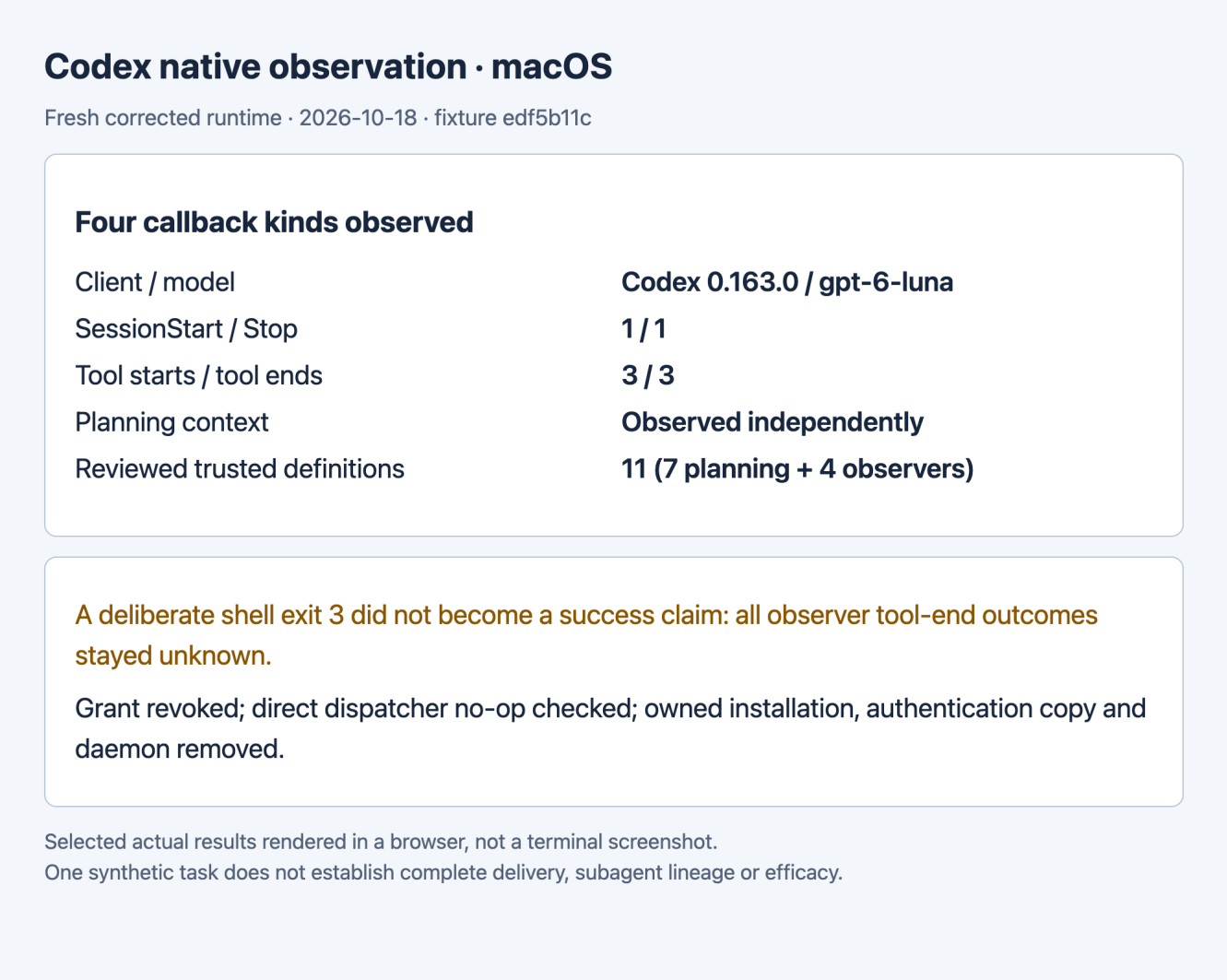
Expected result: One selected client receives observation delivery. Default installations collect nothing; preview and discovery alone do not prove callbacks executed.

Opt in to Codex observations

Scenario: native-observation / workflow-map. 2026-10-18. A committed source, Python 3.11+, Codex and an explicitly authorized disposable project are required. Collection is off by default. Native trust and model access are separate prerequisites.

2. Check actual callback evidence

In an authorized disposable session, exercise a synthetic read/write and an intentional failure. Inspect native records and planning context separately. Retain exact runtime identity and counts; then revoke the grant and remove owned test delivery.



Browser-rendered results from the fresh corrected macOS Codex 0.163.0 / GPT-6-Luna fixture; matching runtime hashes are recorded in the capture manifest.

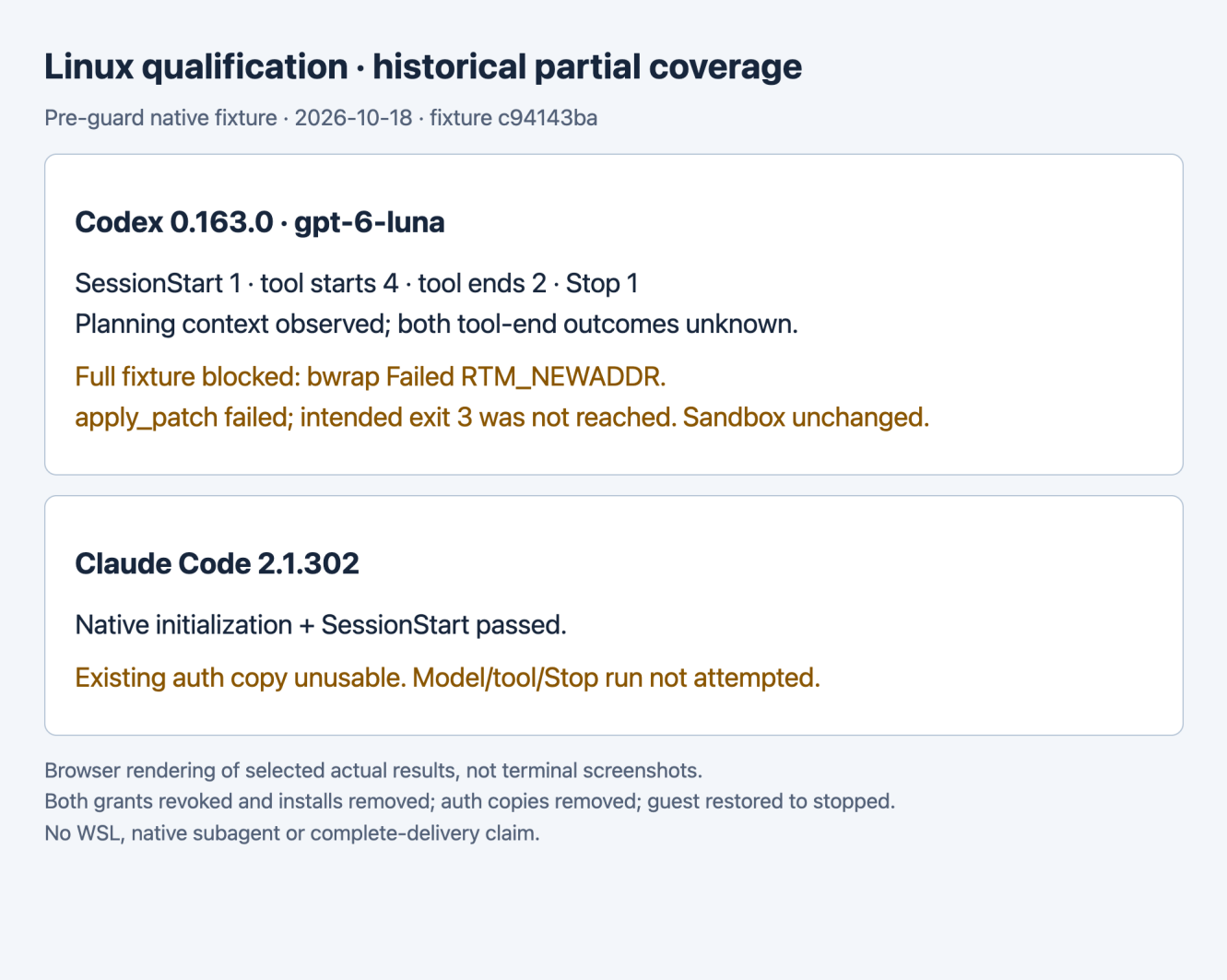
Expected result: All four kinds occurred with the corrected process-directory guard. Tool-end status remains unknown after a nonzero shell result. Revoked no-op is direct-dispatcher proof, not another native model turn. Outside-project rejection has separate focused and installed-dispatcher checks.

Opt in to Codex observations

Scenario: native-observation / workflow-map. 2026-10-18. A committed source, Python 3.11+, Codex and an explicitly authorized disposable project are required. Collection is off by default. Native trust and model access are separate prerequisites.

3. Keep platform blockers visible

Before relying on another host, compare its evidence with the native guide matrix. Treat observed callbacks, usable tools and authenticated model behavior as separate checks. Preserve a blocked result instead of weakening sandbox or authentication boundaries.



Browser rendering of historical pre-guard Linux results: callbacks/planning were observed but Codex tools were blocked; Claude reached initialization/SessionStart only.

Expected result: These historical Linux fixtures do not qualify corrected Linux runtime. Without usable Claude authentication, a model run was not attempted. No WSL, parent lineage, complete delivery or model-effectiveness claim follows.